Practical 3 — Text Preprocessing & TF-IDF

Objective: load a news dataset, clean and lemmatize text, encode labels, and produce a TF-IDF feature matrix. Outputs are saved to `practical3_outputs/`.

In [1]:
```
# Imports and NLTK downloads
import os, re, pickle, urllib.request
import numpy as np
import pandas as pd
from pathlib import Path

import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
import joblib
from scipy import sparse

# GPU device detection (use CUDA if available)
try:
    import torch
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print('device =', device)
except Exception:
    device = 'cpu'
    print('torch not installed; using CPU')

print('libraries ready')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\jenil\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\jenil\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\jenil\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]     C:\Users\jenil\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
device = cuda
libraries ready
```

In [2]:
```
# Download dataset if missing (raw GitHub URL)
DATA_PATH = Path('News_dataset.pickle')
RAW_URL = 'https://raw.githubusercontent.com/PICT-NLP/BE-NLP-Elective/main/3Preproc

if not DATA_PATH.exists():
    print('News_dataset.pickle not found locally — attempting download...')
    try:
        urllib.request.urlretrieve(RAW_URL, str(DATA_PATH))
        print('Downloaded', DATA_PATH)
    except Exception as e:
        print('Automatic download failed:', e)
        print('Please download the file manually from:')
        print(RAW_URL)
        raise

# Load the pickle file (robust loader)
with open(DATA_PATH, 'rb') as f:
    data = pickle.load(f)

print('Loaded dataset type:', type(data))
```

```
Loaded dataset type: <class 'pandas.DataFrame'>
```

```
C:\Users\jenil\AppData\Local\Temp\ipykernel_20656\677916438.py:18: VisibleDeprecatio
nWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=
0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  data = pickle.load(f)
```

In [3]:
```
# Convert loaded object to a DataFrame if necessary
if isinstance(data, pd.DataFrame):
    df = data.copy()
else:
    try:
        df = pd.DataFrame(data)
    except Exception as e:
        # try common structure: (texts, labels)
        if isinstance(data, (list, tuple)) and len(data) == 2:
            texts, labels = data
            df = pd.DataFrame({'text': texts, 'label': labels})
        elif isinstance(data, dict) and 'text' in data and 'label' in data:
            df = pd.DataFrame({'text': data['text'], 'label': data['label']})
        else:
            raise ValueError('Could not convert loaded pickle to a DataFrame')

print('DataFrame shape:', df.shape)
display(df.head())
```

```
DataFrame shape: (2225, 6)
```

	File_Name	Content	Category	Complete_Filename	id	News_length
0	001.txt	Ad sales boost Time Warner profit\r\n\r\nQuart...	business	001.txt-business	1	2569
1	002.txt	Dollar gains on Greenspan speech\r\n\r\nThe do...	business	002.txt-business	1	2257
2	003.txt	Yukos unit buyer faces loan claim\r\n\r\nThe o...	business	003.txt-business	1	1557
3	004.txt	High fuel prices hit BA's profits\r\n\r\nBriti...	business	004.txt-business	1	2421
4	005.txt	Pernod takeover talk lifts Domecq\r\n\r\nShare...	business	005.txt-business	1	1575

In [4]:
```
# Detect text and label columns (common names)
cols_lower = {c: c.lower() for c in df.columns}
text_candidates = ['text','content','article','headline','body']
label_candidates = ['label','labels','category','categories','class','target']

text_col = None
label_col = None
for c in df.columns:
    if cols_lower[c] in text_candidates and text_col is None:
        text_col = c
    if cols_lower[c] in label_candidates and label_col is None:
        label_col = c

# fallback: first object dtype column as text
if text_col is None:
    obj_cols = [c for c in df.columns if df[c].dtype == 'object']
    text_col = obj_cols[0] if obj_cols else df.columns[0]

if label_col is None:
    # try to find a small-cardinality object column
    for c in df.columns:
        if df[c].dtype == 'object' and df[c].nunique() < 1000 and c != text_col:
            label_col = c
            break

print('Detected text column:', text_col)
print('Detected label column:', label_col)
```

```
Detected text column: Content
Detected label column: Category
```

Preprocessing functions

Lowercase, remove punctuation/numbers, remove stopwords, tokenize and lemmatize.

In [5]:
```
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
```

```
def clean_text(s):
    s = str(s).lower()
    s = re.sub(r'http\S+', ' ', s)
    s = re.sub(r'[^a-z\s]', ' ', s)
    s = re.sub(r'\s+', ' ', s).strip()
    return s

def preprocess_to_tokens(s):
    s = clean_text(s)
    toks = word_tokenize(s)
    toks = [t for t in toks if t not in stop_words and len(t) > 1]
    toks = [lemmatizer.lemmatize(t) for t in toks]
    return toks

# Apply preprocessing (may take a few seconds)
df['clean_text'] = df[text_col].apply(clean_text)
df['tokens'] = df['clean_text'].apply(preprocess_to_tokens)
# also create a token-joined string for TF-IDF
df['tokens_joined'] = df['tokens'].apply(lambda toks: ' '.join(toks))

display(df[[text_col,'clean_text','tokens_joined']].head())
```

	Content	clean_text	tokens_joined
0	Ad sales boost Time Warner profit\r\n\r\nQuart...	ad sales boost time warner profit quarterly pr...	ad sale boost time warner profit quarterly pro...
1	Dollar gains on Greenspan speech\r\n\r\nThe do...	dollar gains on greenspan speech the dollar ha...	dollar gain greenspan speech dollar hit highes...
2	Yukos unit buyer faces loan claim\r\n\r\nThe o...	yukos unit buyer faces loan claim the owners o...	yukos unit buyer face loan claim owner embattl...
3	High fuel prices hit BA's profits\r\n\r\nBriti...	high fuel prices hit ba s profits british airw...	high fuel price hit ba profit british airway b...
4	Pernod takeover talk lifts Domecq\r\n\r\nShare...	pernod takeover talk lifts domecq shares in uk...	pernod takeover talk lift domecq share uk drin...

In [6]:
```
# Label encoding (if a label column was detected)
if label_col is not None:
    le = LabelEncoder()
    df['label_str'] = df[label_col].astype(str)
    df['label_encoded'] = le.fit_transform(df['label_str'])
    print('Classes:', list(le.classes_))
    print('Encoded label sample:')
    display(df[[label_col,'label_str','label_encoded']].head())
else:
    print('No label column detected; skipping label encoding')
```

```
Classes: ['business', 'entertainment', 'politics', 'sport', 'tech']
Encoded label sample:
```

	Category	label_str	label_encoded
0	business	business	0
1	business	business	0
2	business	business	0
3	business	business	0
4	business	business	0

TF-IDF vectorization

We vectorize the `tokens_joined` column. Adjust `max_features` / `min_df` as needed for your dataset size.

In [7]:
```
vectorizer = TfidfVectorizer(max_features=10000, ngram_range=(1,2), min_df=2)
X_tfidf = vectorizer.fit_transform(df['tokens_joined'].astype(str))
print('TF-IDF matrix shape:', X_tfidf.shape)
print('Sample features (first 20):', vectorizer.get_feature_names_out()[:20])
```

```
TF-IDF matrix shape: (2225, 10000)
Sample features (first 20): ['aaa' 'aaa title' 'abandoned' 'abba' 'abbas' 'abbasi'
'abbott' 'abc'
 'abiding' 'ability' 'able' 'able access' 'able get' 'able make'
 'able play' 'able see' 'able watch' 'abn' 'abn amro' 'abolish']
```

In [8]:
```
# Save outputs to practical3_outputs/
out_dir = Path('practical3_outputs')
out_dir.mkdir(exist_ok=True)

# cleaned text and token strings
df[['clean_text','tokens_joined']].to_csv(out_dir / 'cleaned_texts.csv', index=Fals
# labels (if present)
if 'label_encoded' in df.columns:
    np.save(out_dir / 'labels.npy', df['label_encoded'].to_numpy())
    joblib.dump(le, out_dir / 'label_encoder.joblib')
# TF-IDF vectorizer + matrix
joblib.dump(vectorizer, out_dir / 'tfidf_vectorizer.joblib')
sparse.save_npz(out_dir / 'tfidf_matrix.npz', X_tfidf)
# save feature names
pd.Series(vectorizer.get_feature_names_out()).to_csv(out_dir / 'tfidf_features.csv'

print('Saved outputs to', out_dir)
print('Files:')
for p in sorted(out_dir.iterdir()):
    print('-', p)
```

```
Saved outputs to practical3_outputs
Files:
- practical3_outputs\cleaned_texts.csv
- practical3_outputs\label_encoder.joblib
- practical3_outputs\labels.npy
- practical3_outputs\tfidf_features.csv
- practical3_outputs\tfidf_matrix.npz
- practical3_outputs\tfidf_vectorizer.joblib
```

Quick checks

Print the number of documents, class distribution (if labels), and a sample cleaned article.

In [9]:
```
print('Documents:', len(df))
if 'label_encoded' in df.columns:
    print('Label distribution:')
    display(df['label_str'].value_counts().head())

print('Sample cleaned text:')
print(df['clean_text'].iloc[0][:1000])
```

```
Documents: 2225
Label distribution:
label_str
sport            511
business         510
politics         417
tech             401
entertainment    386
Name: count, dtype: int64
Sample cleaned text:
ad sales boost time warner profit quarterly profits at us media giant timewarner jum
ped to bn m for the three months to december from m year earlier the firm which is n
ow one of the biggest investors in google benefited from sales of high speed interne
t connections and higher advert sales timewarner said fourth quarter sales rose to b
n from bn its profits were buoyed by one off gains which offset a profit dip at warn
er bros and less users for aol time warner said on friday that it now owns of search
engine google but its own internet business aol had has mixed fortunes it lost subsc
ribers in the fourth quarter profits were lower than in the preceding three quarters
however the company said aol s underlying profit before exceptional items rose on th
e back of stronger internet advertising revenues it hopes to increase subscribers by
offering the online service free to timewarner internet customers and will try to si
gn up aol s existing customers for high speed broadband timewarner also h
```

Conclusion

- The notebook loads `News_dataset.pickle`, performs cleaning, stopword removal, lemmatization and TF-IDF vectorization.
- Outputs are saved to `practical3_outputs/`: cleaned texts, `labels.npy` (if labels), `label_encoder.joblib`, `tfidf_vectorizer.joblib`, `tfidf_matrix.npz`, and `tfidf_features.csv`.

Next steps: tune `TfidfVectorizer` parameters, try n-grams, or use the TF-IDF matrix for classification/clustering experiments.